M7 — Hypergradients, diffusion samplers, a scaling law (N3, N8, O6)

jax-experiments writeup · M7

2026-10-19

Contents

N3 — Learning the learning rate: hypergradients through training **2**

Question . . . 2
The idea . . . 2
The math . . . 2
What we measure and why . . . 3
What I built . . . 3
Results . . . 4
What surprised me . . . 6
Open threads . . . 7

Status: N3 done; N8 and O6 to follow ·
Hardware: Kaggle T4 ×2 (one job per GPU), JAX 0.11.1; N3's constant-LR controls on a Colab T4 (same JAX, Flax and Optax versions); CPU (4 cores) for the tests ·
Data: N3: `data/n3-summary.json` and `data/n3/` (one JSON and log per job; the first run, whose landscape batches were not yet keyed, in `data/n3/first-run/`) ·
Reproduce: N3: `uv run python tools/kaggle/push_r2.py <preset> --push` for `n3b`, `n3c` and `n3d`; the controls are `experiments/n3_hypergrad/run.py --studies reference --constant-lr <1e-3|2e-3> --seed <0|1|2>`; then `uv run python experiments/n3_hypergrad/analyze.py`. CPU tests: `uv run pytest tests/test_n3_hypergrad.py`

N3 — Learning the learning rate: hypergradients through training

Question

N1 found the 4M transformer's best learning rate by search: 18 runs, one per (LR, seed). Training is a differentiable function of its learning rate, so `jax.grad` can, in principle, find that rate by gradient instead. Two questions:

- **Does the hypergradient point at the tuned LR?** Starting from a point in training and looking K steps ahead, which LR does the gradient prefer, and how does it compare with the LR the tuned schedule uses at that point?
- **Can it set the LR online?** Train 3,000 steps with the LR updated by its own hypergradient every K steps, and compare the final bits/byte with the tuned schedule.

The literature predicts a specific failure: **short-horizon bias**. Hypergradients over short horizons favour small, greedy learning rates, because a small step reduces the loss soonest even when a larger one would reach a lower loss later (Wu et al. 2018).

The idea

Training is a function, so it has a derivative. K steps of AdamW map $(\theta_0, \eta_1, \dots, \eta_K)$ to θ_K. The validation loss after them, $L_{\text{val}}(\theta_K)$, is a scalar function of the learning rates, and reverse mode differentiates it through every step: through the forward and backward passes of each step (a second derivative of the model), through Adam's moments and bias correction, through the clipping and the weight decay.

The meta-loss. Its gradient with respect to $\log \eta$ (log, so that a step changes the LR by a factor) is the hypergradient. Its zero crossing is the LR that minimises the loss K steps later: the *K-step-greedy* LR.

What it costs. Reverse mode through K steps stores what each step's backward pass needs. With `jax.checkpoint` around every step, it stores only the carry (θ, m, v: three copies of the parameters) per step and recomputes the rest, so memory grows with K times 50 MB for the 4M model rather than K times the activations.

The math

One AdamW step, as ml-ladder's protocol runs it (clipping at global norm 1, $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, decoupled weight decay $\lambda = 0.1$):

$$g \leftarrow g \min\left(1, \tfrac{1}{\|g\|}\right), \quad m \leftarrow \beta_1 m + (1-\beta_1) g, \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2,$$

$$\theta \leftarrow \theta - \eta \left(\frac{m/(1-\beta_1^t)}{\sqrt{v/(1-\beta_2^t)} + \epsilon} + \lambda\theta \right).$$

The hypergradient for the segment's shared LR is

$$\frac{\partial L_{\text{val}}(\theta_K)}{\partial \log \eta} = \sum_{k=1}^{K} \eta_k \frac{\partial L_{\text{val}}(\theta_K)}{\partial \eta_k},$$

the sum of the per-step hypergradients; the code computes both. Each $\partial\theta_K/\partial\eta_k$ propagates through the later steps' Jacobians $\partial\theta_{j+1}/\partial\theta_j$, which contain the Hessian of the training loss. That is where the second derivative enters.

Why a short horizon prefers a small LR. On a quadratic valley with curvature h, gradient descent at rate η shrinks the error along it by $(1-\eta h)$ per step. Over a short horizon the loss is dominated by the highest-curvature directions, which a large η overshoots (or destabilises), while the low-curvature directions, where a large η pays, contribute only over many steps. The greedy optimum therefore sits below the long-run optimum, and the shorter the horizon, the further below.

What we measure and why

- **Equivalence (CPU).** The differentiable AdamW equals the training optimiser (`jaxexp.train.optimizer(`
step for step. The hypergradient equals a central finite difference (float64), and remat changes nothing.
- **The reference.** The tuned protocol in this setting: 3,000 steps of 64 × 256 bytes, peak 8e-3 (the best LR at T=256 in ml-ladder's `c-gpu-lr2-256.json` and in O3), 5% warmup, cosine to 10%, fp32. It saves checkpoints at steps 500 and 1,500.
- **The landscape.** From step 0 and the two checkpoints, for K = 10, 30 and 100: the meta-loss after K steps at nine constant LRs (2.5e-4 to 6.4e-2, factors of 2) and its hypergradient. The same K training batches and the same 64 held-out windows are used at every LR, so the curve is a deterministic function of the LR; they are keyed by (seed, start, K). The K-step-greedy LR is the hypergradient's zero crossing; it is compared with the schedule's mean LR over the same K steps.
- **Online.** 3,000 steps with no schedule. Every K = 30 steps, the hypergradient of the meta-loss after that segment (fresh held-out windows each time) updates $\log\eta$ by Adam (step 0.05). It starts from 1e-3 (too small), 8e-3 (the tuned peak) and 3.2e-2 (too large). A segment whose loss or hypergradient is not finite is undone and the LR halved. The runs from 1e-3 and 8e-3 are repeated with each hypergradient clipped to ± 1 before Adam sees it. Final bits/byte is on the same 262,144 validation bytes as every other run.
- **The control.** The same training stream at a constant 1e-3 and 2e-3, with no hypergradient: does the adaptation beat leaving the LR alone?
- **Held-out windows.** The meta-objective's windows come from the validation split after its first 1,000,000 bytes, so they never overlap the bytes on which bits/byte is reported.

What I built

- **`jaxexp.hypergrad`:**
  - `adamw_init` and `adamw_step`: ml-ladder's AdamW written out so the LR is a traced input. Its square root has a zero derivative at 0 (see below).
  - `make_unroll`: K steps in `lax.scan`, with `jax.checkpoint` on each step.

  - `make_meta_loss`: the validation loss after K steps, as a function of $\log \eta$ (one value, or one per step).
- **`experiments/n3_hypergrad/run.py`** (studies `reference`, `landscape`, `online`; `--clip-hypergrad` for the online runs, `--constant-lr` for the control), `analyze.py`, and the Kaggle presets `n3b`, `n3c` and `n3d`.
- **Tests:** `tests/test_n3_hypergrad.py`, 4 tests.
  - **A NaN the tests found.** The attention key bias has a gradient of exactly zero, because softmax ignores a shift shared by all keys. Its Adam second moment is then exactly 0, the derivative of $\sqrt{v}$ there is infinite, and reverse mode multiplies it by zero: NaN in every hypergradient. The fix is a square root whose derivative at 0 is 0; the forward value is unchanged.
  - **Finite differences on the transformer agree only to 0.13%.** Even in float64, the transformer's meta-loss is not smooth below about 10^{-9}, so the exact check uses a small attention-free model (agreement 3.5×10^{-9}). On the transformer, the test checks that reverse and forward mode agree.

Results

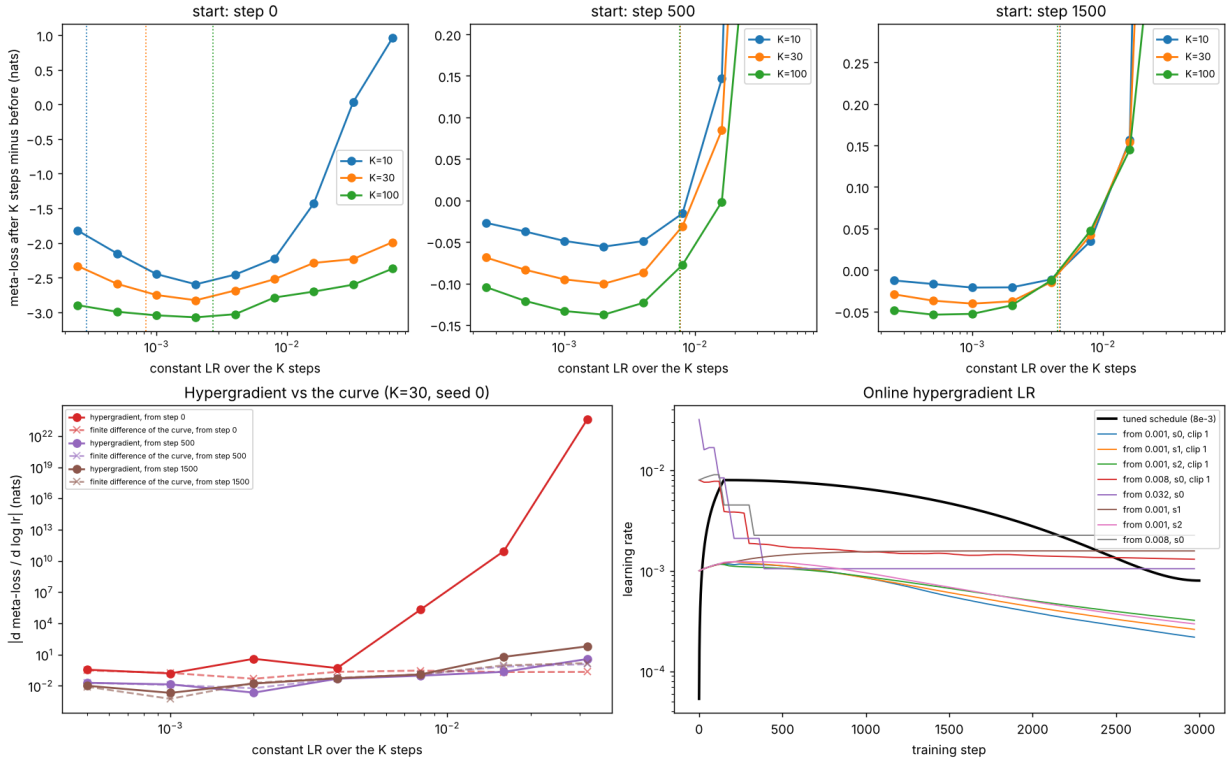


Figure 1: N3: the meta-loss after K steps at constant LRs from three points of the tuned run (seed 0; dotted: the schedule's mean LR over those steps); the hypergradient against the curve's finite differences; and the online LR trajectories.

The reference reproduces ml-ladder. The tuned schedule (peak 8e-3) reaches 1.6338 bits/byte (1.6303–1.6358) in fp32, against ml-ladder's 1.6308 (1.6239–1.6339) in fp16 PyTorch at the same LR. It trains at 4.8–5.5 steps/s.

The K-step-greedy LR (the meta-loss curve's minimum). Medians over seeds 0–2, with min–max. The greedy LR is the vertex of a parabola in $\log \eta$ through the grid minimum and its neighbours; no minimum lies at the grid's edge.

start	K	greedy LR	schedule's mean LR	greedy / schedule	hypergradient matches the curve up to	explodes from
0	10	1.7e-3 (1.6–2.0e-3)	2.9e-4	5.97 (5.61–6.88)	1e-3	1.6e-2 – 6.4e-2
0	30	1.8e-3 (1.8–1.8e-3)	8.3e-4	2.18 (2.15–2.18)	5e-4 – 1e-3	8e-3
0	100	1.8e-3 (1.7–1.9e-3)	2.7e-3	0.66 (0.65–0.69)	5e-4 – 1e-3	2e-3 – 4e-3
500	10	1.9e-3 (1.7–2.0e-3)	7.7e-3	0.25 (0.22–0.26)	4e-3 – 8e-3	3.2e-2 – none
500	30	1.5e-3 (1.3–1.7e-3)	7.7e-3	0.20 (0.17–0.22)	4e-3 – 8e-3	1.6e-2 – 6.4e-2
500	100	1.7e-3 (1.6–1.8e-3)	7.7e-3	0.22 (0.21–0.23)	2e-3	8e-3 – 1.6e-2
1500	10	1.2e-3 (1.1–1.4e-3)	4.7e-3	0.26 (0.24–0.29)	8e-3	3.2e-2 – none
1500	30	1.0e-3 (0.91–1.0e-3)	4.6e-3	0.22 (0.20–0.22)	8e-3	3.2e-2 – 6.4e-2
1500	100	7.2e-4 (6.4–9.3e-4)	4.5e-3	0.16 (0.14–0.21)	2e-3 – 8e-3	1.6e-2 – 3.2e-2

"Matches" means within half the curve's central difference (or 0.02 nats near zero), at every grid LR up to that one. "Explodes" means the hypergradient is not finite or exceeds 10 in magnitude, where the curve's own slope is below 1. "None" means it never exploded on the grid.

- **Short-horizon bias, as predicted.** Once warmup is over, the greedy LR is a fifth to a quarter of what the tuned schedule uses at that point (0.16–0.26 by median). The greedy choice is not wrong over its own horizon: over the K steps, it lowers the held-out loss by 0.02 to 0.07 nats more than the schedule's LR does. The schedule's large LR pays only over the whole run.
- **A longer horizon does not close the gap.** From step 1500 the ratio falls from 0.26 at K = 10 to 0.16 at K = 100. Holding a constant LR for longer favours a smaller one (it lowers the noise floor). Horizons of 10–100 steps never reach the regime where a large LR wins.
- **At initialisation it is the other way round.** Over the first 10 steps the greedy LR is 6× the warmup's: warmup is cautious, not greedy. By K = 100 the ratio is 0.66.
- **The hypergradient has a trust region.** From steps 500 and 1500 it matches the curve up to 4e-3–8e-3 for $K \leq 30$, and up to 2e-3–8e-3 at K = 100. From step 0 it matches only up to 5e-4–1e-3, below the greedy LR itself. Above that it disagrees, even in sign (seed 1, step 0, K = 10, at 2e-3: -0.69 against the curve's $+0.12$), and then it explodes: from 2e-3 at K = 100 from step 0, and from 8e-3 or more after warmup. The meta-loss itself stays smooth throughout. Its zero crossing would have put the greedy LR at 3.9e-3 for that seed, against the curve's 1.7e-3.

Online. Every 30 steps, Adam (step 0.05) updates $\log \eta$ with the segment's hypergradient. 3,000 steps; bits/byte on the same 262,144 validation bytes.

start LR	hypergradient	seeds	bits/byte	LR at steps 150 / 1,500 / 2,970
1e-3	clipped to ± 1	0, 1, 2	1.8043 (1.8031–1.8210)	1.2e-3 / 5.6–6.7e-4 / 2.2–3.2e-4
1e-3	raw	0, 1, 2	1.7880 (1.7746–1.8388)	see below
8e-3	clipped to ± 1	0	2.3581	3.9e-3 / 1.5e-3 / 1.3e-3
8e-3	raw	0	2.2609	4.5e-3 / 2.3e-3 / 2.3e-3
3.2e-2	raw	0	3.0003	8.4e-3 / 1.1e-3 / 1.1e-3
constant 1e-3 (no hypergradient)	—	0, 1, 2	1.8145 (1.8042–1.8229)	—
constant 2e-3 (no hypergradient)	—	0, 1, 2	1.7604 (1.7461–1.7773)	—
tuned schedule (peak 8e-3)	—	0, 1, 2	1.6338 (1.6303–1.6358)	8.0e-3 / 4.7e-3 / 8.0e-4

- **Hypergradient descent learns to anneal, greedily.** With clipping, the LR rises from 1e-3 to about 1.2e-3 in the first 150 steps. It then falls steadily, to 5.6–6.7e-4 at step 1,500 and 2.2–3.2e-4 at the end: a decay schedule, discovered online. It ends 0.17 bits/byte behind the tuned schedule, whose LR is 3–8× higher throughout.
- **And it gains nothing over leaving the LR alone.** Seed by seed, the clipped online run from 1e-3 minus a constant 1e-3 on the same training stream is -0.002, -0.011 and $+0.000$ bits/byte. A constant 2e-3 beats every online run (1.7461–1.7773). Rising a little and then annealing, it ends where it would have ended had it never moved.
- **Raw hypergradients break the meta-optimiser rather than the model.** One exploding value (10^{13} to 10^{27}) fills Adam's second moment, and every later update of $\log \eta$ is then about zero: the LR freezes. Seed 1 (raw, from 1e-3) met one value of -539 at step 210; its LR then crept up to 1.6e-3 and stayed there, and it finished at 1.7746, *better* than any clipped run, and level with a constant 2e-3 on that seed (1.7773). Seed 2 never exploded and annealed like the clipped runs (1.7880). Seed 0 is the first run's: it never exploded, annealed to 1.8e-4 and finished at 1.8388 (`data/n3/first-run/`).
- **Starting at the tuned peak, without warmup, is worse than starting low.** From 8e-3 the hypergradients explode at once (47 of 100 segments raw, 53 clipped). The guard halves the LR twice in the first 300 steps (two segments undone), and the run never recovers: 2.26–2.36 bits/byte, worse than any run from 1e-3. From 3.2e-2: 3.00.
- **Cost.** An online run took 3.9× the reference's wall time (2,175–2,318 s against 544–624 s): each segment differentiates through 30 remat'd steps, then evaluates 64 held-out windows.

What surprised me

- **The hypergradient goes wrong before it blows up.** I expected it either to work or to overflow. Instead, from step 0 it has the wrong size, and sometimes the wrong sign, at LRs where the meta-loss curve is perfectly smooth. Its trust region there lies *below* the greedy

LR. Reverse mode multiplies 10–100 Jacobians of the training map. Once their product grows, the derivative stops describing the function, long before it overflows.

- **Longer horizons made the greedy LR smaller, not larger.** The textbook story says short horizons favour small LRs and longer ones relax towards the tuned value. Here, for a constant LR held over the horizon, the optimum fell from 1.2e-3 to 7.2e-4 between K = 10 and K = 100 (from step 1500).
- **The damage was done to the meta-optimiser, not the model.** One hypergradient of 10^{27} is harmless to the weights (the LR step is bounded), but it fills Adam's second moment for the rest of the run (it decays by 0.999 per update; the run has 100 updates). The run that froze by accident at 1.6e-3 beat every run whose LR kept adapting.
- **The first 10 steps want 6× the warmup's LR.** Measured greedily, warmup looks too timid. But the run that started at the tuned peak with no warmup never recovered (2.26 bits/byte), so a greedy 10-step view of warmup misses what it protects against.

Open threads

- **Forward mode over the whole run.** With one or two hyperparameters (peak LR, warmup length), a JVP through all 3,000 steps costs a few training runs and no extra memory. It has no horizon bias at all, but the same Jacobian product grows inside it: would it explode the same way?
- **Unbiased estimators for long horizons.** Persistent evolution strategies (Vicol et al. 2021) smooth the meta-loss instead of differentiating it, and avoid the exploding product. The landscape here, with its known greedy optima, is a ready-made test bed.
- **The per-step hypergradient** along the tuned schedule was recorded at every start (`schedule_hypergrad_per_step`) and is not analysed here. It says which steps of the schedule want more or less LR.
- **The clipping threshold** (± 1) was not tuned, and nor was the meta step (0.05). The greedy annealing, though, looks like a property of the objective rather than of the optimiser: the landscape's greedy LR falls along the run too (K = 30: 1.5e-3 from step 500, 1.0e-3 from step 1500).